NYPC Master Qualification Round Replay

팀명	NOJIWOONG
사용한 LLM의 제공자(플랜)	Claude Max (5x), Claude Pro, Codex Pro
사용한 LLM 모델	Claude Fable 5, Opus 4.8, Sonnet 5, GPT 5.5

※ 구독제가 아닌 API 사용 방식을 사용했다면, 어떤 방식으로 사용했는지 적어주세요.
여러 종류를 사용했다면 전부 적어주세요.

주의 사항

- 리플레이를 **반드시 이 양식에 따라 작성해야 할 필요는 없습니다**. 원하는 대로 양식을 수정하거나 새롭게 양식을 만들어도 무방합니다.
- **리플레이 제출 시 내용 작성, 문법 교정 등을 포함하는 LLM 사용이 일체 금지됩니다.**
- 제출 시에는 **파일 > 내보내기**를 통해 **PDF 형식으로 제출**해주세요.

리플레이는 Qualification Round의 평가 기준에 포함되지 않으나, 제출한 내용에 이상이 있거나 부정행위의 소지가 발견될 경우 운영본부에서 재제출을 요청할 수 있습니다. 또한 리플레이를 제출하지 않을 경우 Final Round 진출이 불가합니다.

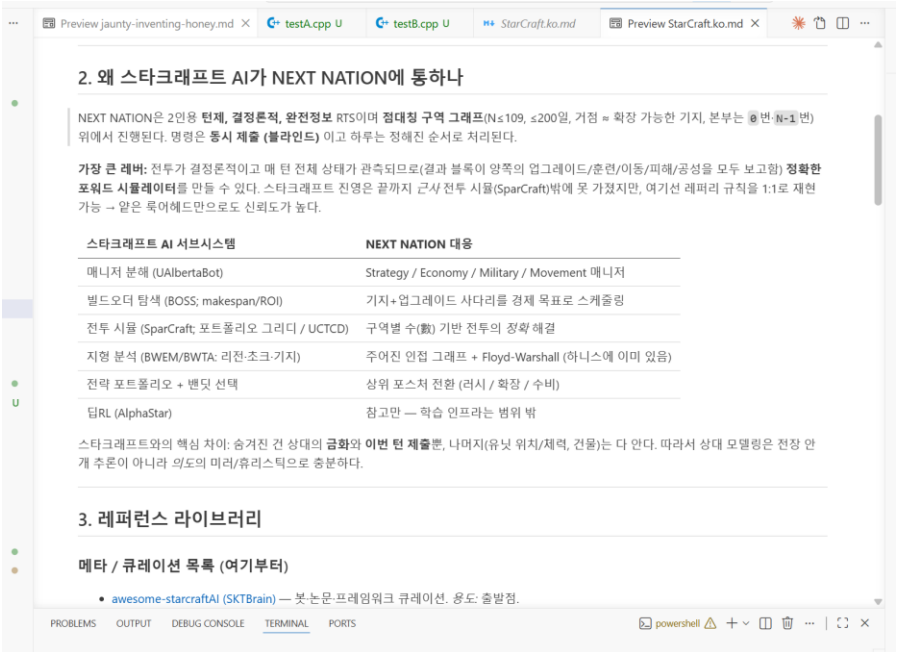
자세한 내용은 대회 규칙 또는 문제의 리플레이 제출 페이지를 참고해 주세요.

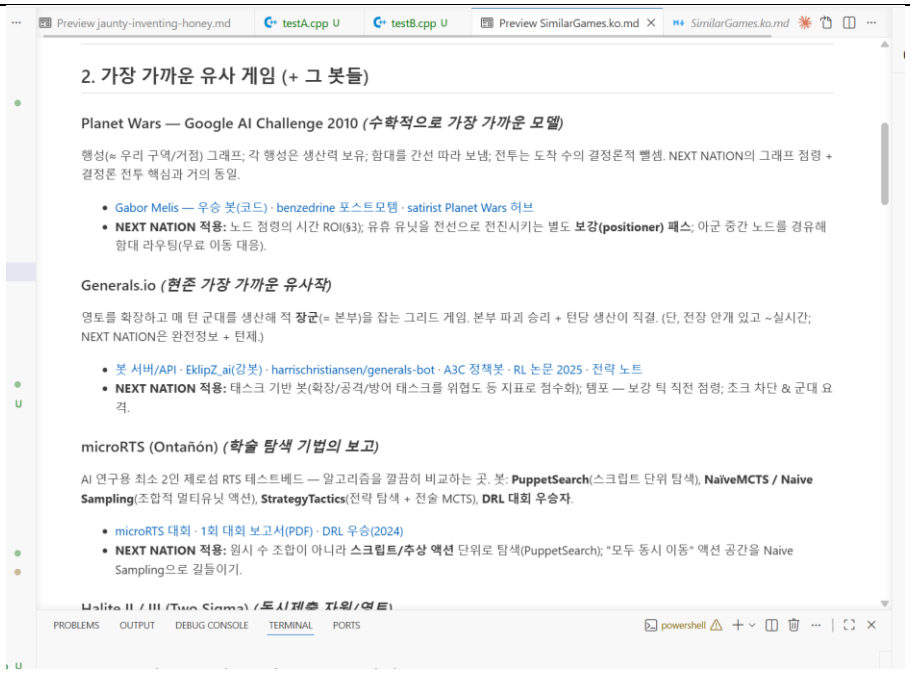
문제 풀이

팀원들 간의 역할 배분:

- HongJungi: 팀장 및 태스크 정리, 전략 구현
- NOJIWOONG: 중간 평가 로그 분석, 승패 요인 분석 및 핵심 전략 수립
- JangSeongho: 코드 버그 마이너 패치 및 기능 개선, 확장
- labyrinth: 각종 파라미터 머신러닝 기반 최적화

#HongJungi

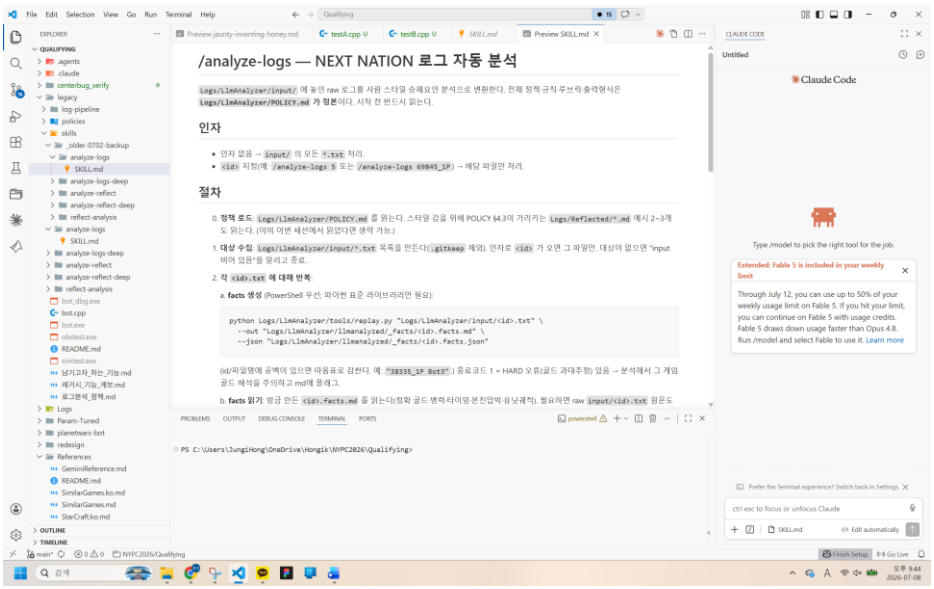
중간 평가 시각	6월 29일 21:00:00
제출 시각	6월 29일 19:34:34
내용	**발견한 문제점** ● 게임의 복잡도가 매우 높아보임 ● 게임 구조 상 기존의 훌륭한 전략이나 잘 만들어진 AI 레퍼런스가 존재할 것으로 예상 **LLM 활용 방식** ● 게임 규칙, 이미지와 함께 유사한 종류의 게임 레퍼런스를 찾는데에 활용함 **접근 방향** ● 스타크래프트와 매우 유사한 종류의 전략게임이라고 판단하여 스타크래프트 AI 레퍼런스를 찾고 LLM을 통해 정리, 및 구조에 맞게 유사한 종류의 코드를 작성하였음 **자료** 



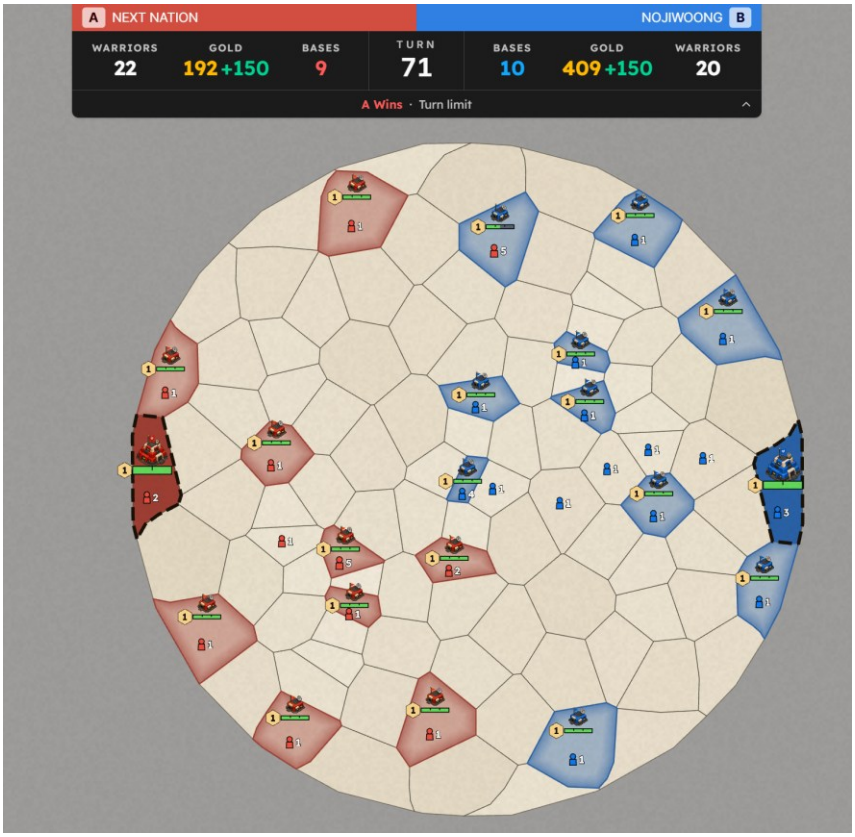
한계점 및 보완점

- 규칙과 시각적인 부분 실제로 유사한 레퍼런스의 게임은 planet wars뿐이었고, 현재의 코드는 스타크래프트를 베이스로 하였으나, llm은 스타크래프트를 권장하지 않았음

중간 평가 시각	7월 1일 15:00:00
제출 시각	7월 1일 13:06:48
내용	**발견한 문제점** ● 승리하고 패배하는 규칙만 알 뿐 승 패 요인을 정확히 판단하지 못하였음 **LLM 활용 방식** ● 이 때 까지는 단순히 구상한 전략을 코드로 옮기는 번역기로만 활용 ● 로그를 분석하는데에 활용하려 하였음 **접근 방향** ● 직전까지의 중간평가에서 패배했던 판들의 로그를 살펴보며 주로 승패가 갈리는 규칙을 확인하였음 ● 봇, 중간평가 모두 포함하여 패배요인을 인간이 정성적으로 분석하여 md형식으로 정리하였음. **한계점 및 보완점** ● 로그를 분석함에 있어 별다른 지시사항이나 명확한 프롬프트 없이 단순 로그 입력만으로는 LLM이 규칙에 대한 이해도가 낮고, 로그를 읽는 방법론적인 부분에서 자연어 입력이 아닐 때 파싱 능력이 떨어짐

중간 평가 시각	7월 2일 10:00:00
제출 시각	7월 2일 05:19:06
내용	**발견한 문제점** ● 모든 로그를 사람이 분석하는데에 한계가 있음. 일정 조율부터 시작해서 간신히 만나더라도 공통된 분석과 해결책을 다음, 여유있게 다다음 중간평가 전까지 반영하기는 쉽지 않음. **LLM 활용 방식** ● Claude Skills를 활용하여 인간의 로그 분석 방식을 학습한 정책대로 매 중간평가 때마다 대량의 로그를 분석, 해결책 제시를 지시 ● 제시된 해결책을 현행의 코드에 반영하여 로그와 같은 패배요인이 생기지 않도록 하였음 **접근 방향** ● 직전까지의 중간평가에서 패배했던 판들의 로그를 살펴보며 주로 승패가 갈리는 규칙을 확인하였음 **자료**  **한계점 및 보완점** ● 단순히 경기 결과만을 기반으로 분석을 수행하기 때문에 로그의 각종 상황들이 코드에서 왜 그렇게 판단했는지에 대한 이해가 부족함. 또한 파서의 한계로 인간 수준의 분석과 해결책, 대비책 제시가 불가능함

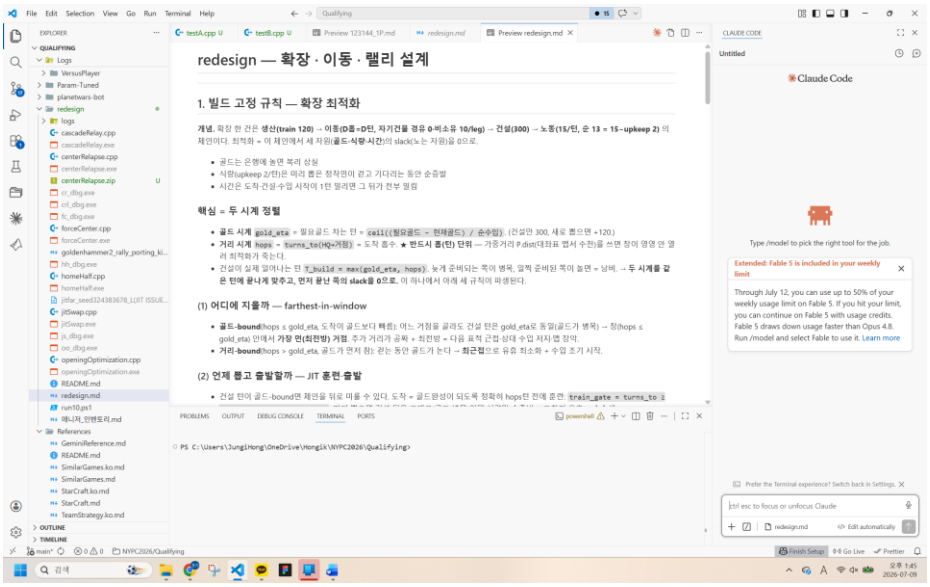
중간 평가 시각	7월 3일 15:00:00
제출 시각	7월 3일 14:49:06
내용	**발견한 문제점** ● 위에서 작성한 로그분석 skills는 오로지 로그만을 기반으로 분석하기 때문에 현행 코드의 원인분석을 할 수 없음 **LLM 활용 방식** ● Skills를 수정하여 현행 코드를 기반으로 분석하도록 수정 **자료**  **한계점 및 보완점** ● 앞에서와 마찬가지로 facts의 한계로 인해 실제로 인간의 로그만큼 훌륭한 분석이 나오지 않는다.

중간 평가 시각	7월 5일 10:00:00
제출 시각	7월 5일 09:01:34
내용	**발견한 문제점** ● 랠리포인트공격 **LLM 활용 방식** ● 게임 상황에 대한 문답을 통한 전략 구상 ● 구상한 전략을 알고리즘 형태로 구조화한 뒤 코드로 구현 **접근 방향** ● 패배 로그 중 퍼포먼스 포인트가 높았던 상대와의 전투에서 상대의 일관적인 패턴을 발견(특정 마릿수만큼의 유닛이 함께 이동하며 기지를 분쇄) **자료** 

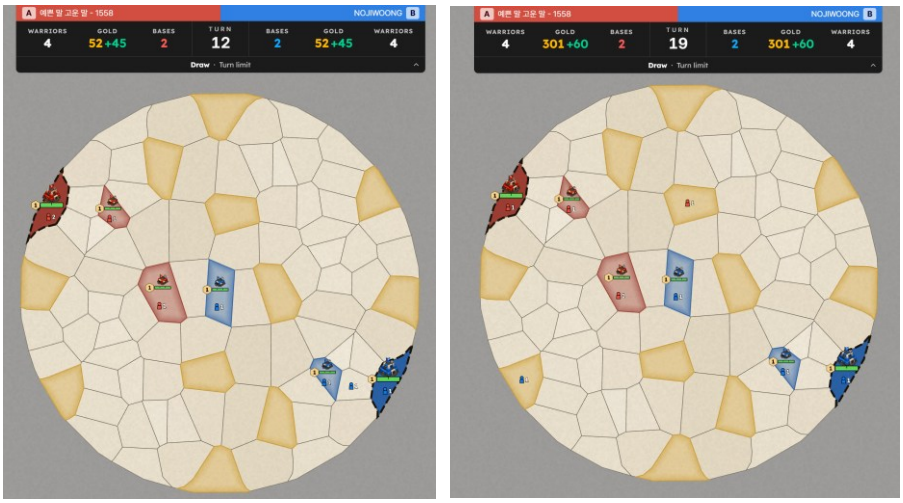
중간 평가 시각	7월 7일 15:00:00
제출 시각	7월 7일 13:17:40
내용	**기지 확장 최적화 및 이동 방식 최적화** **발견한 문제점** ● 게임 규칙에서 활용하는 이동 방식은 유클리드 거리를 기반으로 하기 때문에 최소 턴 수 이동이 아님. ● 게임 규칙 상 유닛의 이동이 아군의 거점을 경유하는 경우 이동과 노동의 시간이 분배 되어있기 때문에(분배 되어있지 않더라도 동일함) 목적지까지 가는 데에 유닛만 바꿔치기 시켜서 출발시키면 경유지를 찍고 가는 한 턴을 아낄 수 있음. ● 매일 유닛의 보급 비용으로 2골드가 소모되기 때문에 기지 확장 건설 시 거점 도착과 동시에 300골드가 모이도록 유닛을 생산하면 늦게 생산한만큼 유닛의 보급 비용을 절약할 수 있음. **LLM 활용 방식** ● 구상한 아이디어를 규칙에 대조하여 합당성이 있는지 판별 ● 아이디어가 코드로 동작하기 위한 순서도를 작성하고, 특히 기초적인 생산, 이동 관련 메커니즘을 수정하는 것이기 때문에 현행 코드와의 병목점, 충돌점 등을 면밀이 분석함 **접근 방향** ● 해당 기능만을 위한 코드를 순차적으로 작성하였고, 이후 현행 코드에서 계산하는 방식을 해당 기능으로 대체하였음. ● 이동 방식에 대한 기능을 cascadeRelay, 기지 건설 최적화 기능을 JIT Train이라고 지었음. ● 특히 cascadeRelay의 경우 모든 유닛 이동에 사용할 수 있고, 사용할 수 있도록 하기 위하여 게임 시작 시 맵에 대해 모든 거점끼리의 이동 턴 수, 거점 경유 시 이동 턴 수를 계산하여 유닛 이동 명령이 해당 계산 보드 위에서 이루어지도록 하였음.

자료

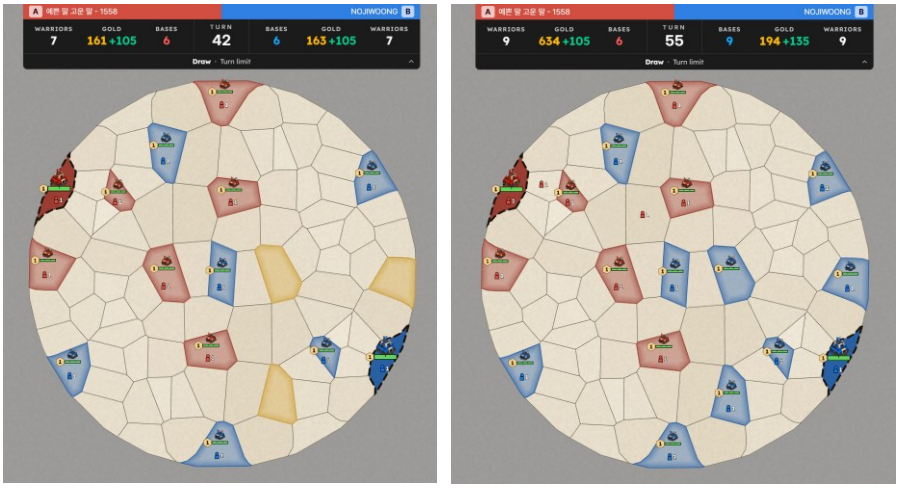
- 알고리즘 설계


redesign — 확장 · 이동 · 랠리 설계
1. 빌드 고정 규칙 — 확장 최적화
2. 이동 턴 계산 규칙 — 실제 동선 ≠ 최단 hops, 내 이동 vs 상대 예측의 비대칭
Claude Code

- 구현 결과 1. 동시 생산 선제 출발 및 같은 턴 원거리 거점 도착 (cascadeRelay의 결과)



- 구현 결과 2. JIT Train으로 같은 기지 수, 같은 병력 수, 2골드 우위 또한 상대보다 많은 거점 확보 가능

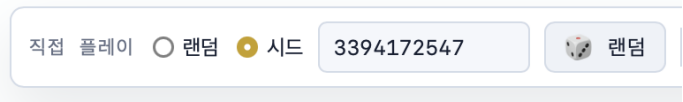
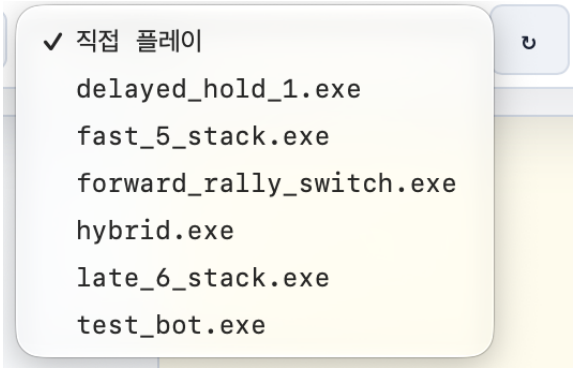
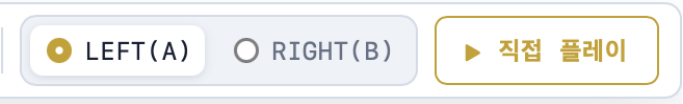


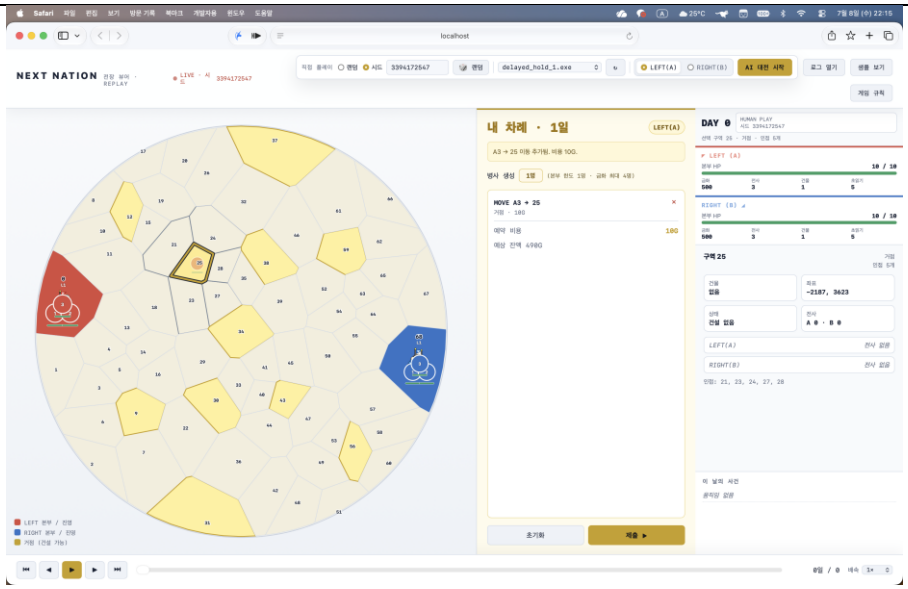
한계점 및 보완점

- 모든 유닛의 이동 방식을 수정하는 방향의 기능 추가인 탓에 스파게티처럼 꼬여버린 코드 상에서 수정하기 매우 복잡함. 단순한 기능 추가로 인해 대부분의 로직에서 버그가 터져 나옴
- 현재 마지막 제출 코드에서 확장 스케쥴링에 식비를 절감하고 더 가까운데에 지음으로써 한 턴 아끼는 로직을 추가하지 않았음. 발생할 경우 치명적이지만 20판 중 한 판 꼴로 나타나는 현상이기 때문에 우선도 낮음으로 판단하여 다른 문제 해결에 집중하였음.

제출 번호	98565
제출 시각	7월 7일 22:49:39
내용	**발견한 문제점** ● 7월 7일 21시 중간평가에서 중앙 기지가 파괴 당한 이후 중앙 거점을 먹기 위한 로직이 텅 빈 중앙에 이유 없이 유닛을 집결시킴 ● 마찬가지로 해당 평가에서 초반에 유닛을 뽑아 상대 본진으로 쳐들어와서 끝내는 일명 러쉬 전략에 4패를 기록함 **LLM 활용 방식** ● 중앙으로 유닛을 보내는 핵심 로직을 코드에서 어느 부분이 담당하고 있는지 탐색하였음. (현재 코드를 기준으로 if war를 판별하는 부분) **접근 방향** ● 중앙 기지로 집결하는 로직을 latch형태로 교체하여 해당 로직이 특정 조건 이후 재발동 하지 않도록 수정하였음. ● 러쉬봇들은 공통적으로 본진 공격 이후 계획 없이 작동하는 봇이기 때문에 단순히 약간의 차이가 있는 러쉬봇들에 대하여 공통적으로 대응할 수 있는 전략을 수립하고, 상대의 러쉬 행동 감지 시 기존의 전략이 아닌 러쉬 대응 전략 수행 이후 러쉬가 끝났다 판단하면 기존 전략으로 회복해서 정상작동하도록 하였음. **LLM 한계점 및 보완점** ● 러쉬 전략 대응을 위한 아이디어 구상에서는 활용할 수 없었음.

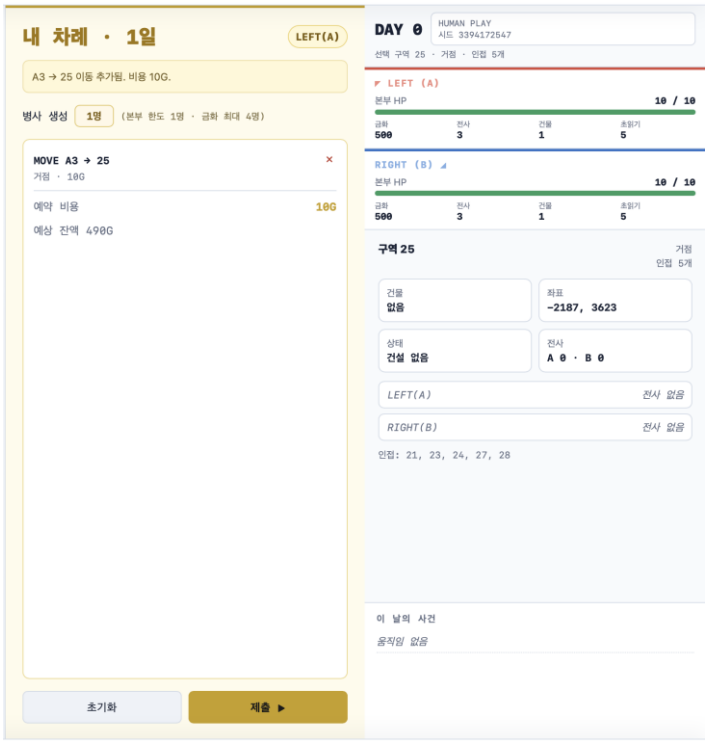
#JangSeongho

중간 평가 시각	중간 평가 X (장성호 GUI 툴)
제출 시각	제출 X
내용	연습게임인 버섯게임을 통해 실제로 게임을 진행할 수 있는 GUI툴이 없을 경우 게임 규칙 공개와 동시에 실제 플레이 가능한 툴을 만들고자 시도했음, 이때 버섯게임의 경우 1P, 2P의 유불리가 존재했기에 양 진형에서 모두 플레이 할 수 있게 설계 - 기능 1. 시드 값 고정 및 랜덤 사용가능 (대회 규칙의 시드 생성룰과 일치)  2. 우리가 제작한 cpp 코드를 컴파일한 exe 파일을 상대로 배틀가능  3, 진형 선택  4. 게임규칙 보기  5. 전체 적인 UI



아래 화면과 같이 직접 전사를 클릭하고 이동 하는 등의 실제 게임플레이 가능

6. 실제 게임에 사용되는 데이터들을 확인 할수 있음



Day1에 해당 GUI툴을 완성함으로써 팀원전체의 전반적인 규칙에 대한 이해 및 전략을 실제 테스트해볼 수 있는 환경을 마련.

대회 일정 후반부에는 실제 중간평가를 통해 취약했던 부분을 테스트 해보면서 기존 봇을 강화하는 등 직접 손으로 테스트 해보면서 사용

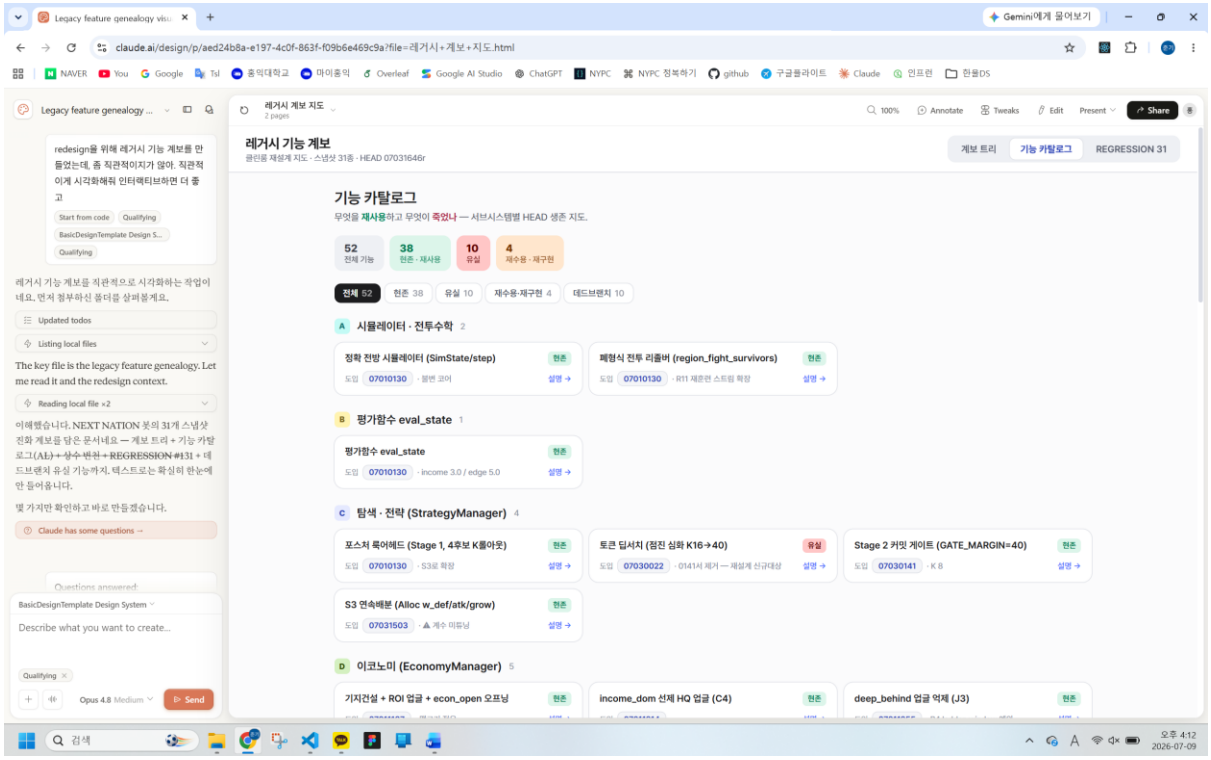
중간 평가 시각	7월 1일 21:00:00 (장성호 8승봇)
제출 시각	7월 1일 20:11:55
내용	**발견한 문제점** ● 우선 기존 채점시 사용되는 샘플AI를 모두 이겨서 8승을 기록하자 라는 기조를 가지고 기존 제출 코드를 개선시작 **LLM 활용 방식** ● 로그를 실제눈으로 보면서 확인했을때 의도하지 않은 움직임, 기물 생성, 이동방향에 대해서 LLM의 코드 분석기능을 통해서 해당 움직임이 버그인지, 아니면 여러 전략들이 맞물리며 생긴 일종의 버그인지 판단하고 자했음 ● 특히, 3번, 5번 AI에게 계속 패배하게 되며 해당 부분을 codex와 opus4.8을 통해서 브레인스토밍을 계속 진행함 **접근 방향** ● 해당 봇들을 돌파하는데 특정한 파라미터값을 도입하고 제출을 반복하면서 조금씩 LLM을 통해 코드를 수정해 가는 방향으로 진행했음 **LLM 한계점 및 보완점** ● 특정파라미터를 통해 하드코딩을 진행하여 추후 수정 필요

중간 평가 시각	중간평가 X (장성호 러쉬대응 1~10)
제출 시각	7월 2일 00:03:33 이후
내용	**발견한 문제점** ● 특정 파라미터를 이용하여 하드코딩한 이유로 러쉬 전략 및, 공격 중 전사가 한마리씩 이동하는데 이때 상대 기지를 지나가는 상황에서 상대 기지를 지나며 HP가 모두 소멸하는 이슈 **LLM 활용 방식** ● 중간평가로 얻은 로그 데이터를 이용해서 예를 들어 63턴의 이 움직임이 일어나게 된 이유가 코드의 어떤부분인가 등등의 식으로 실제 내가 원하지 않은 동작이 일어난 이유를 분석 **접근 방향** ● 파라미터값을 수정해가면서 위 이유들에 대응하기 위해 노력 **LLM 한계점 및 보완점** ● 여러 회차들을 시도해봤으나 별다른 수확이 없었음

중간 평가 시각	7월 6일 21:00:00 및 7월 7일 10:00:00 (장성호 golden hammer)
제출 시각	7월 6일 20:27:36
내용	**발견한 문제점** ● 랠리포인트의 위치선정 및 약간의 공격성 이슈 **LLM 활용 방식** ● 직전 제출들에서 뚜렷한 개선의 아이디어가 떠오르지않아 LLM들에게 현재 코드의 약점들을 제시해달라고 한뒤 해당 약점들을 읽어보고 우리 코드의 강화할 방안 등을 알려달라 한뒤 각각의 LLM들과 직접적으로 의견을 주고 받으며 수정해야할점의 아이디어를 제공받음 ● 위 내용을 바탕으로 우리의 최근 승리 및 패배 로그들을 모두 모아서 LLM과 함께 랠리포인트의 선정에 대한 아이디어를 제공받고 위를 수정하기로 했음 **접근 방향** ● 실제 우리가 지거나 이겼던 로그 리플레이를 보면서 이 위치로 전사들을 모아 공격하는게 좋지 않았을까? 라는 점들을 계속 생각하고 해당 위치를 코드로 구현하기 위해 규칙성을 찾는 느낌으로 진행함 **LLM 한계점 및 보완점** ● 해당코드엔 캐스케이드 릴레이가 완벽하게 적용되어 있는 상태가 아니기때문에 캐스케이드 완성시 해당 코드와 합쳐야함

제출 번호	98199, 98205, 98243, 98250
제출 시각	7월 7일 22:05:40, 22:06:49, 22:11:05, 22:12:36
내용	**발견한 문제점** ● 7월 7일 21:00:00 중간 평가에서 rush형태에 대응을 전혀 하지 못해 4패 가량을 기록해서 rush대응을 위해 직접 rush전략을 개발 **LLM 활용 방식** ● 우리 봇을 상대로 승리했던 rush 전략들을 4가지 타입으로 로그 분석을 통해 분류 하고 rush전략의 경우 비교적 간단한 전략에 가깝기에 해당 로그를 역으로 분석해 직접 4가지 형태의 rush 봇을 제작 **접근 방향** ● rush의 타입 분류 및 구현

코드 기능 카탈로그



코드 점검 및 개선 과정에서 현재의 코드에서 갖고 있는 기능들을 정리하고, 기능 간 연결점, 병목점 등을 분석하기 위하여 Claude Design 기능을 활용하여 기능 카탈로그를 만들었음.

#labyrinth

NEXT NATION AI Bot 개발 보고서

구글의 딥마인드, 알파스타와 같은 몬테카를로 기반 강화학습을 시도하였다.

v1 - 초기 보상 설계 v1에서는 명령을 내리면 +점수 기지를 지으면 +점수처럼 행동 자체에 점수를 주고 있었다. 초반 점수 설계를 우리가 만들어둔 휴리스틱 코드에 적용해 보았더니 총점은 +0.074, 반대로 의미 없는 명령을 남발하면 게임당 최대 +3.0으로 보상 설계에 오류가 있다는 것을 발견했다. 봇 입장에서는 이기는것보다 명령을 많이 내는것이 이득이다 라고 판단하고 이동만 많이해 점수만 챙기는 현상이 발생했다.

v2 - 그래서 보상 설계를 포텐셜 기반으로 변경했다. 또한, 승패만으로 이루어진 보상은 게임이 끝나야만 알 수 있어서 학습이 느리다는 문제가 있다. 그래서 매턴 지금 형세를 판단해 학습을 시키되 이 형세 판단이 게임 전체로 보면 서로 상쇄되어 승패만 남도록 하는 방식을 사용했다.

보상 = (최종 승패) + β × (γ·형세(다음) − 형세(현재))

v3 - 보상을 수정했더니 봇이 아무것도 안하는게 제일 안전하다고 학습해버렸다. 모든 평가 경기에서 건물 업그레이드가 0회 초반에 상대방 기지로 러쉬를 들어간 후 아무행동도 하지 않는 문제가 생겼다. 이를 수정하기 위해 잘하는 휴리스틱 봇을 옆에 선생님처럼 배치하고 그 봇들의 결과를 일부분 따라하도록 만들었다. 하지만, 선생님 모델을 붙이고도 문제가 발생했다. 초반 러쉬를 들어가는 현 상황은 이미 모든 병력을 러쉬에 투입했으므로 돈이 나올 곳이 없었다. 이러한 상황에서 선생님 모델도 할 수 있는 행동은 가만히 있기 밖에 없었다.

v4 - 이대로는 안되겠다고 판단해서 "강화학습" 말그대로 강화를 하고자 모델을 사전학습 하는 방식을 도입했다. 잘하는 봇을 흉내 내게 하기 "Behavior Cloning" 방식을 도입했다. 우리가 만든 휴리스틱 코드중 가장 좋은 코드를 이용해 학습 데이터를 생성했다. 기존에 만들어둔 다양한 코드가 있었기 때문에 해당 모델들과 대국을 하며 다양한 학습 데이터를 만들 수 있었다. 150개의 맵에서 3,150경기 데이터를 모았다. 이 데이터를 바탕으로 신경망을 학습시켰다. 사전학습 결과는 상당히 좋았다. 기존에는 한 번도 하지 않던 기지 업그레이드를 모방했으며 교사가 업그레이드한 자리를 신경망이 정확히 맞힐 확률은 82%로 학습이 잘 되었다는 걸 알 수 있었다.

v5 - 오히려 기존의 보상대로 학습을 이어서 했더니 모델이 붕괴되는 현상을 발견했다. 설정해놓은 보상이 적절하지 않다고 결론지었고 보상을 더 적절하게 설정할 수 없다라고 판단해 복잡한 형세 판단을 다 버리고 승/무/패 에만 점수를 주었다.

승에 +1 무승부에 -0.1 패에 -1점을 부여했다.

그리고 자기 자신, 과거 자기 자신, 휴리스틱 코드와 수많은 대국을 펼쳤다. 200판쯤 돌렸을때는 성능 향상이 보였다. 하지만 그 뒤로는 엔트로피가 1.3→2.4로 폭증하며 제대로 된 학습이 진행되지 않는 모습을 보이며 붕괴되는 모습을 보였다. 여기서 현재의 인프라와 제한 시간 내에는 강화학습 방법으로는 불가능하다고 판단하여 포기했다.

후기

- **HongJungi:** 작년 2025 코드배틀에서 자력으로 본선을 나가봤고, 본선에서 아무런 기반 지식(코드, 알고리즘, 프로그래밍 언어) 없이 마카롱만 먹다 온 것이 후회돼 다시 한번 참가 신청을 하였다. 올해 다시 대회를 준비하면서 여러가지 알고리즘과 최적화 이론을 공부하였고, 이번 예선 과정에서는 특히 LLM을 단순히 생각-코드 번역기로 활용한 것이 아닌, 문서 정렬, 생각 구조화, 태스크 정리 등 다양한 업무 방식을 활용해가며 사용하였기에 LLM 활용 방법을 조금 더 적응하여 사용할 수 있게 되었다.

- **JangSeongho:** LLM을 통해 내가 부족한 지식을 명확히 하고 특히 코드를 분석할때 LLM을 적극활용하여 비전공자임에도 코딩 그 자체보다 전략을 구상하고 테스트 하고 수정을 하는 것에 더 집중할 수 있었다.

- **NOJIWOONG:** 전략가로서 게임을 분석하며 떠올린 추상적인 전략들을 LMM을 통해 코드로 젼역하고 테스트할 수 있었던 점이 재밌었습니다. 코딩 문법에 얽매이지 않고 전략의 본질에만 집중할 수 있어 문제 해결 속도가 빨라던 것 같습니다. 패배했던 로그를 뜯어보며 우리 봇의 약점을 찾아내고 극복하능 과정에서 성취를 느꼈습니다. 하지만 한계도 경험했습니다. LMM은 복잡한 거리 계산이나 데미지 시뮬레이션 등을 빠르게 처리하며, 명형 하나하나나 매 턴 단위의 분석의 능력은 뛰어났습니다. 반대로 그렇게 만들어진 행동들이 모여 형성하는 전체적인 흐름이나 전략적 시퀀스를 LMM한테 인지시키는 데에 애를 먹었습니다. 예를 들어 "상대가 특정 건물을 짓지 않고 병력만 생산한다면 올인 러쉬다"와 같이 거시적 맥락은 스스로 파악하지 못했습니다. 이를 통해 전체적인 맥락을 설계하는 것은 전략가의 역량이며 LMM의 한계는 전략가가 가진 게임 이해도와 통찰력의 깊이에 의해 정해진다는 것을 느꼈습니다.

- **labyrinth:** 지금까지 강화학습을 쉽게 생각하고 있었지만, 실험을 진행하며 보상 설계가 봇의 행동을 완전히 다르게 만들 수 있다는 것을 알게 되었다. 의도와 다르게 행동하는 봇을 보며 보상 체계를 계속 수정하고, 사전학습 같은 새로운 방법도 시도해보았다. 모델이 점점 나아지는 모습을 보이기도 했지만 결국 안정적인 성능까지는 이르지 못한 채로 실험을 마무리하게 되었다. 이번 프로젝트를 통해 보상 설계와 강화학습의 불안정성을 직접 경험했고 이를 통해 머신러닝에 대한 시야를 확장할 수 있었다.